Chapter 4 worked solutions

Self-Attention from Vectors to Matrices

These written solutions cover Exercises 1, 2, 4, 5, and 6. The companion notebook covers Exercises 3 and 7–12. Unless an exercise supplies different shapes, the calculations use the chapter's three positions in source order and the value vectors $(1, 0)$, $(0, 2)$, and $(1, 1)$. The formulas keep exact values throughout. Decimals are rounded for display.

1. A query with equal scores

Replace the first query with $\mathbf{q}'_1 = (0, 0)$. A dot product multiplies corresponding feature coordinates and adds the products. Both coordinates of this query are zero, so its dot product with each key is zero.

$$\begin{aligned}\mathbf{q}'_1 \cdot (1, 0) &= 0(1) + 0(0) = 0,\\ \mathbf{q}'_1 \cdot (0, 1) &= 0(0) + 0(1) = 0,\\ \mathbf{q}'_1 \cdot (1, 1) &= 0(1) + 0(1) = 0.\end{aligned}$$

The score row remains $(0, 0, 0)$ after dividing by $\sqrt{2}$. Softmax exponentiates every score and divides by the total for this query. Each exponential is $\exp(0) = 1$, so the denominator is $1 + 1 + 1 = 3$. The attention weights are $(1/3, 1/3, 1/3)$.

Key position	Value	Weight	Weighted value
1, *the*	$(1, 0)$	$1/3$	$(1/3, 0)$
2, *red*	$(0, 2)$	$1/3$	$(0, 2/3)$
3, *car*	$(1, 1)$	$1/3$	$(1/3, 1/3)$

Adding the first coordinates gives $1/3 + 0 + 1/3 = 2/3$. Adding the second gives $0 + 2/3 + 1/3 = 1$. The first output is $\mathbf{o}'_1 = (2/3, 1)$. Equal weights make this read the ordinary average of the three value vectors.

A finite score of zero contributes one to the exponential sum. Its weight depends on the competing scores in the same row. Here all scores are equal, so all three keys receive the same weight. The two other queries retain their original scores and outputs because softmax uses a separate denominator for each query.

The same uniform result would follow from any constant row of finite scores. If the common score were c, each key would get $\exp(c)/(3\exp(c)) = 1/3$. In a stable softmax, subtracting the row maximum would give three zeros again. In both calculations the normalized weight records the key's share among its competitors.

2. Change what is read

Keep the original first query $(1, 0)$ and replace the third value $(1, 1)$ with $(3, -1)$. The score row is still $(1, 0, 1)$ before scaling. The third key has its original coordinates $(1, 1)$, so it still matches the first query with a dot product of one. In particular, the new second coordinate of the third **value** never enters the dot product.

Set $x = 1/\sqrt{2}$ and let $u = \exp(x)$. The first weight row remains (a, b, a), where

$$a = \frac{u}{2u+1} \approx 0.40111209, \qquad b = \frac{1}{2u+1} \approx 0.19777581.$$

Key position	Value used now	Contribution to the first output
1	$(1, 0)$	$(a, 0)$
2	$(0, 2)$	$(0, 2b)$
3	$(3, -1)$	$(3a, -a)$

Sum each coordinate separately. The first gains $a + 3a$ from positions 1 and 3. The second gains $2b$ from position 2 and loses a from position 3.

$$\begin{aligned} \mathbf{o}'_1 &= (a, 0) + (0, 2b) + (3a, -a) \\ &= (4a, 2b - a) = \left(\frac{4u}{2u+1}, \frac{2-u}{2u+1}\right) \\ &\approx (1.60444837, -0.00556046). \end{aligned}$$

Since $u \approx 2.02811498$, the second coordinate is slightly negative. This lies between -1 and 2, the minimum and maximum of the new second coordinates in V.

The input value matrix V changes in its third row. The query and key matrices, their raw dot-product matrix $QK^\top$, the scaled score matrix, and the entire weight matrix A stay fixed. The output matrix $O = AV$ changes because every query that gives the third key positive weight now reads a different value at that position. For any query i, the change in its output is $A_{i3}[(3, -1) - (1, 1)] = A_{i3}(2, -2)$. For the first query this is $(2a, -2a)$, which gives the same answer when added to the original output $(2a, 2b + a)$.

4. Use two sequence lengths

Imagine four decoder positions reading six encoder positions. There are four requests, and each request compares with six keys. Query and key rows have three features apiece, so each comparison is a sum of three products. The result is a grid with four query rows and six key columns.

Quantity	Shape	What its axes represent
Q	4×3	Query positions, matching features
K	6×3	Key positions, matching features
$K^\top$	3×6	Matching features, key positions
$QK^\top$ and $S = QK^\top/\sqrt{3}$	4×6	Query positions, key positions
$A = \operatorname{softmax}_{row}(S)$	4×6	Query positions, key positions
V	6×5	Key positions, value features
$O = AV$	4×5	Query positions, value features

In $QK^\top$, the inner dimensions of $(4 \times 3)(3 \times 6)$ match because queries and keys share a three-coordinate feature space. The divisor is $\sqrt{3}$ because each dot product has three terms. Neither scaling nor softmax changes the shape. Softmax divides across the six entries in each of the four query rows.

In AV, the inner dimensions of $(4 \times 6)(6 \times 5)$ match because every weight column has a corresponding value row. Consider the output at query position 4 and value coordinate 5. It is $O_{4,5} = \sum_{j=1}^{6} A_{4j} V_{j5}$. This sum uses the fourth query's weights to combine the fifth coordinate of the six values. The other four coordinates use the same weights and a different column of V. Combining all five coordinates gives the output vector at query position 4. Changing the number of keys would change how many values enter each sum, while the four requests would still produce four output vectors.

5. Inspect directionality

In the toy example, the first query and first key are both $(1, 0)$, and the third query and third key are both $(1, 1)$. The dot product in either direction is one. The scaled scores S_{13} and S_{31} are therefore both $x = 1/\sqrt{2}$.

To turn a score into a weight, compare it with every score in the **same query row**. Let $u = \exp(x)$. The two weights have the same numerator, u, but the rows they come from have different denominators.

Query	Scaled row	Exponential row	Row denominator
1	$(x, 0, x)$	$(u, 1, u)$	$D_1 = 2u + 1$
3	$(x, x, 2x)$	(u, u, u^2)	$D_3 = 2u + u^2$

$$A_{13} = \frac{u}{2u+1} \approx 0.40111209, \qquad A_{31} = \frac{u}{2u+u^2} \approx 0.24825508.$$

In the first row, the third key ties for the highest score. In the third row, the first key has the same score x, but the third key now has the higher score $2x$. The third key's comparison with itself is $(1,1)\cdot(1,1)=2$ before scaling. Its exponential u^2 raises the denominator of the third row. The first key therefore receives a smaller share of query 3's total than the third key receives of query 1's total. The exact ratio is $A_{13}/A_{31}=D_3/D_1\approx 1.61572563$.

More generally, if finite scores satisfy $S_{ij}=S_{ji}$, the exponentials also match. The two weights agree precisely when the corresponding row denominators agree. Symmetry of the raw scores describes pairwise comparisons. Symmetry of the normalized weights would additionally require those comparisons to have equal shares in their respective rows. The first and second query rows in this example have the same multiset of scores, $(x,0,x)$ and $(0,x,x)$, so their denominators agree. That gives $A_{12}=A_{21}=1/(2u+1)$. These entries remain equal even though the matrix as a whole is asymmetric.

6. Mask at the wrong stage

The first causal query is allowed to use position 1. Its two future keys are positions 2 and 3. Start from the scaled score row $(x,0,x)$ with $x=1/\sqrt{2}$, and again write $u=\exp(x)$. If we apply softmax with all three positions present, the result is

$$\alpha_1=\frac{(u,1,u)}{2u+1}=(a,b,a)\approx(0.40111209,0.19777581,0.40111209).$$

Now erase the future weights without changing the surviving weight. The row becomes $(a,0,0)$ and its sum drops to $a\approx 0.40111209$. The other $1-a\approx 0.59888791$ of the original weight has been discarded. The resulting output is

$$a(1,0)+0(0,2)+0(1,1)=(a,0)\approx(0.40111209,0).$$

The future values contribute zero, but the first value has been scaled by a fraction. Its weight was computed using a denominator that included both future keys.

For the correct causal calculation, exclude the two future scores before normalization. Represent them by $-\infty$, whose exponential is zero. The score row $(x,-\infty,-\infty)$ then gives

$$\frac{(u,0,0)}{u+0+0}=(1,0,0),\qquad \mathbf{o}_1^{causal}=1(1,0)=(1,0).$$

With only one permitted key, that key receives the entire weight. When several keys are permitted, their exponentials share a denominator made from just those keys. In exact arithmetic, dividing the erased row by its surviving total would produce the same weights as excluding the future scores before softmax, provided at least one key is allowed. In this example that division is $(a,0,0)/a=(1,0,0)$. The step omitted by the exercise's incorrect procedure is the redistribution of the full unit weight among the allowed keys.

The general calculation follows the same pattern. Let J be the set of allowed key indices and D the sum of exponentials over all key indices. For each surviving weight $\exp(S_{1j})/D$, the surviving total is $\sum_{k\in J}\exp(S_{1k})/D$. Dividing the former by the latter cancels D and leaves the denominator $\sum_{k\in J}\exp(S_{1k})$. Masking the scores first lets softmax compute this denominator directly, with the maximum taken over the allowed scores in its stable form.